



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 12

Team No.: 4

Project Title: Linux File Descriptor and File I/O Management System

Team Members

Roll Number	Student Name	Signature
2520080066	Amrutha Varshini J	
2520090125	V. Hasith Varma	
2520090132	Keerthana G	

1. Abstract

Our project, **Linux File Descriptor and File I/O Management System**, is designed to demonstrate how Linux manages files through file descriptors and system calls. In Linux, files are treated as resources accessed through integer file descriptors, and operations such as opening, reading, writing, seeking, and closing are performed through system calls. The project will provide a terminal-based interface through which users can perform basic file operations while observing the underlying Linux mechanisms. The implementation will use C programming on Ubuntu/Linux and will primarily involve system calls such as `open()`, `read()`, `write()`, `lseek()`, and `close()`, along with APIs such as `stat()` for obtaining file information. The project will also demonstrate file permissions, file offsets, error handling, and the relationship between user programs and kernel-managed file resources. The expected outcome is a working Linux application that provides hands-on understanding of file descriptors, file I/O, and system calls while clearly demonstrating Operating Systems and systems programming concepts.

2. Problem Statement

In Linux, file operations are performed through system calls and file descriptors that provide a controlled interface between user programs and the operating system kernel. Understanding how these mechanisms work is important for learning Operating Systems and Systems Programming, but they are often studied only theoretically. This project aims to develop a terminal-based Linux system that demonstrates file creation, opening, reading, writing, appending, file-pointer movement, and closing using Linux system calls. The project will allow users to observe how file descriptors are assigned and used and how the operating system manages file access. The implementation will focus on Linux/POSIX mechanisms rather than GUI, web, database, or CRUD-based application development.

3. Objectives

1. To understand and demonstrate the concept of **file descriptors** and their role in Linux file management.
2. To implement basic file operations using Linux system calls such as `open()`, `read()`, `write()`, `lseek()`, and `close()`.
3. To demonstrate file metadata, permissions, file offsets, and error handling using Linux/POSIX APIs.
4. To develop a working terminal-based system that provides practical understanding of Linux file I/O and the interaction between user programs and the operating system kernel. (List the specific objectives of the project.)

4. Proposed Methodology

The project will be implemented as a menu-driven command-line application in C on Ubuntu/Linux. The application will provide options for creating or opening files, reading data, writing data, appending data, moving the file pointer, displaying file information, and closing files. Each operation will be implemented using appropriate Linux system calls rather than high-level file-handling functions wherever applicable.

When a file is opened, Linux returns a file descriptor that will be maintained by the application. This descriptor will be used for subsequent read, write, seek, and close operations. The `lseek()` system call will be used to demonstrate movement of the file offset, while `stat()` will provide file metadata such as size and permissions. Appropriate error checking will be included to handle invalid file names, permission errors, and unsuccessful system calls. The complete application will be compiled using GCC and built using a Makefile.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
open()	Opens an existing file or creates a new file and returns a file descriptor
read()	Reads data from a file using its file descriptor
write()	Writes data to a file using its file descriptor
close()	Releases an open file descriptor
lseek()	Changes the current file offset for random/sequential access
stat()	Retrieves file metadata such as size, permissions and timestamps
File Descriptors	Provides a kernel-managed identifier for an opened file
File Permissions	Demonstrates Linux access control for files
File Offset	Demonstrates the current position used during file I/O
errno / Error Handling	Identifies and handles system-call failures
POSIX/Linux File I/O	Demonstrates the interface between user programs and kernel file management
Kernel–User Space Interaction	Demonstrates how system calls provide controlled access to OS resources

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520080066	Amrutha Varshini J	Design and integration of the overall system; implementation of file opening/creation, file descriptor management, and integration of modules
2520090125	V. Hasith Varma	Implementation of read, write, append and file-pointer operations using Linux system calls; testing of file I/O functionality
2520090132	Keerthana G	Implementation of file metadata, permissions and error handling; test cases, documentation and output verification

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system and execution environment
C	Primary programming language for system-level implementation
GCC	Compiling the C source code
Linux/POSIX System Calls	File operations and interaction with OS resources
Make / Makefile	Automated project compilation
VS Code / Vim	Source-code development and editing
Git	Version control and project collaboration
GitHub	Source-code repository and project submission

8. Expected Outcome

The project is expected to produce a working command-line Linux application that demonstrates file descriptor and file I/O management using Linux system calls. Users will be able to create or open files, read and write data, append content, move the file pointer, view file information, and close files. The application will display appropriate success messages, file descriptors, file metadata, and error messages for invalid operations. The final implementation will demonstrate practical understanding of file descriptors, system calls, file offsets, permissions, error handling, and Linux kernel interaction.

9. GitHub Repository

GitHub Repository Link: <https://github.com/amrutha-0123/Operating-Systems-and-Systems-Programming>

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____